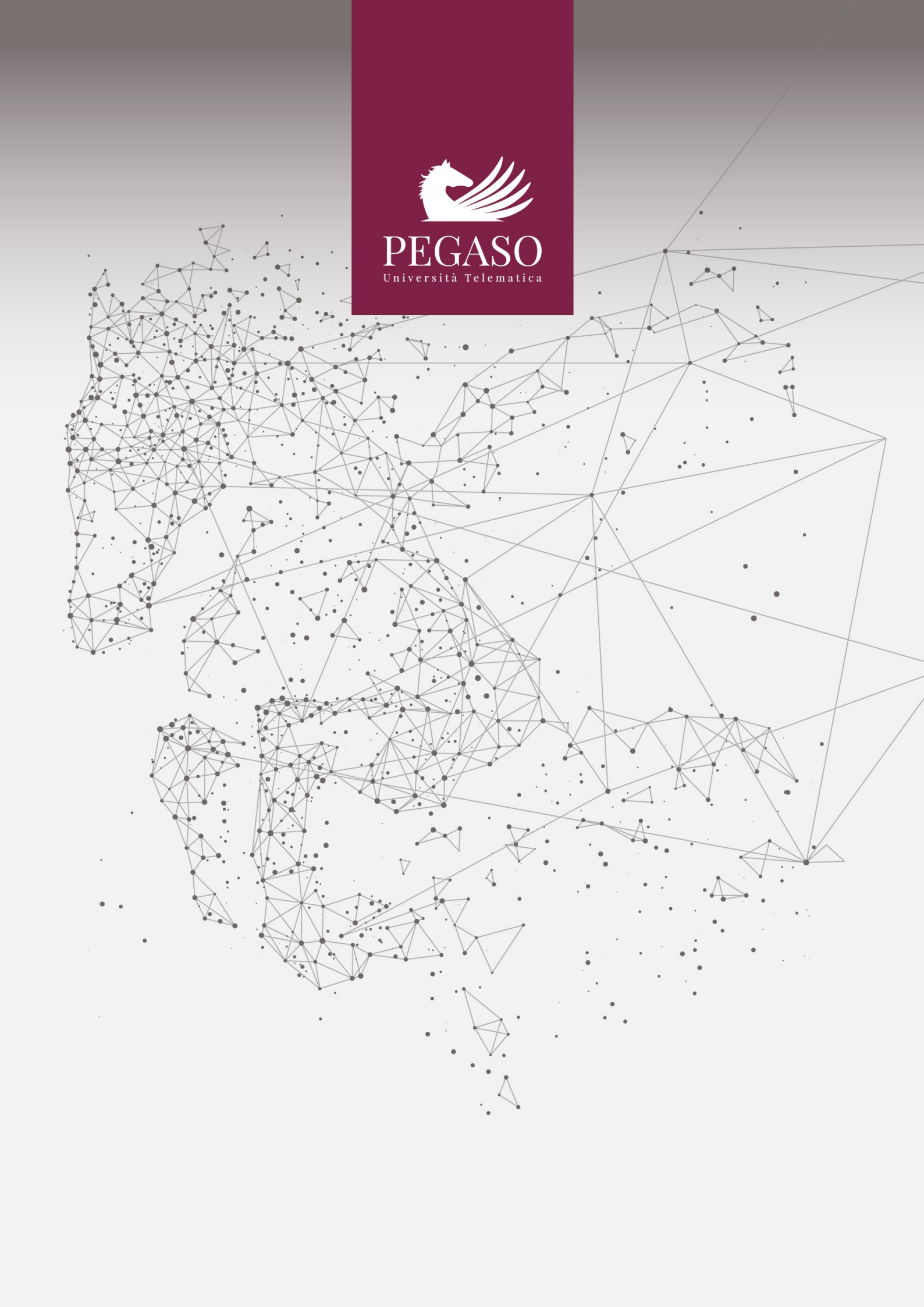

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ISTRUZIONI ASSEMBLY MISPS PER TRASFERIMENTO DI BYTE E HALF WORD.....3

2. FORMATO E INDIRIZZAMENTO PER TRASFERIMENTO DI BYTE E HALF WORD.....6

3. TIPI DI DATO PRIMITIVI PER GLI INTERI.....10

BIBLIOGRAFIA.....16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Istruzioni Assembly MISPS per trasferimento di Byte e half word

Le applicazioni che richiedono l'elaborazione di testi sono molto diffuse, quindi risulta utile disporre di istruzioni Assembly per la gestione di sequenze di 8 bit e 16 bit, lunghezze utilizzate dalla codifica ASCII a 7 e 8 bit e dagli standard Unicode UTF-8 e UTF-16.

I Codici Operativi delle istruzioni Assembly MIPS che coinvolgono operandi di tali lunghezze usano il suffisso *b* per operandi di 8 bit, riferito al termine *Byte*, ed il suffisso *h* per operandi di 16 bit, riferito al termine *half word* (mezza parola).

Il **Linguaggio Assembly** MIPS include istruzioni di trasferimento dati *Load* e *Store* per sequenze di 8 bit e 16 bit. Descriviamo sinteticamente queste istruzioni poiché condividono la Sintassi, il Formato e la Modalità di Indirizzamento delle istruzioni *lw* e *sw* relative a sequenze di 32 bit:

LOAD per Byte e *half word*

- **lb** (*load Byte*) Esempio: lb $t0, 50 ($s1)

legge gli 8 bit contenuti in una locazione di Memoria come un intero con segno e li scrive negli 8 bit meno significativi dei 32 bit del registro $t0 effettuando l'*estensione del segno* sui bit rimanenti (posti a 1 se il bit di segno è 1, a 0 se il bit di segno è 0).

- **lbu** (*load Byte unsigned*) Esempio: lbu $t0, 50 ($s1)

legge gli 8 bit contenuti in una locazione di Memoria come un intero *unsigned* e li scrive negli 8 bit meno significativi dei 32 bit del registro $t0 *ponendo a* 0 *tutti i bit rimanenti*.

- **lh** (*load half word*) Esempio: lh $t0, 50 ($s1)

legge i 16 bit contenuti in due locazione di Memoria consecutive come un intero con segno e li scrive nei 16 bit meno significativi dei 32 bit del registro $t0 effettuando l'*estensione del segno* sui bit rimanenti (posti a 1 se il bit di segno è 1, a 0 se il bit di segno è 0).

- **lhu** (*load half word unsigned*) Esempio: lhu $t0, 50 ($s1)

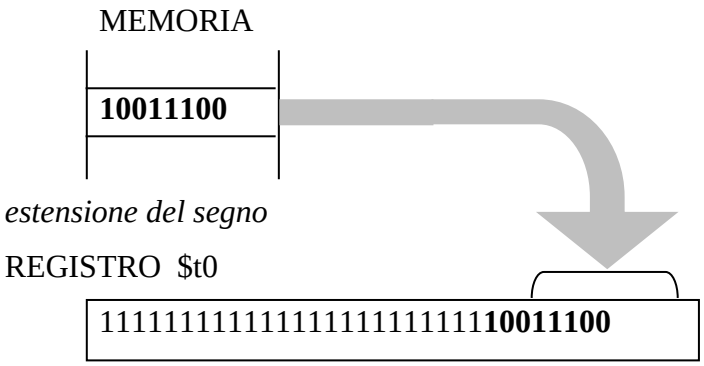
legge i 16 bit contenuti in due locazioni di Memoria consecutive come un intero *unsigned* e li scrive nei 16 bit meno significativi dei 32 bit del registro $t0 *ponendo a* 0 *tutti i bit rimanenti*.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

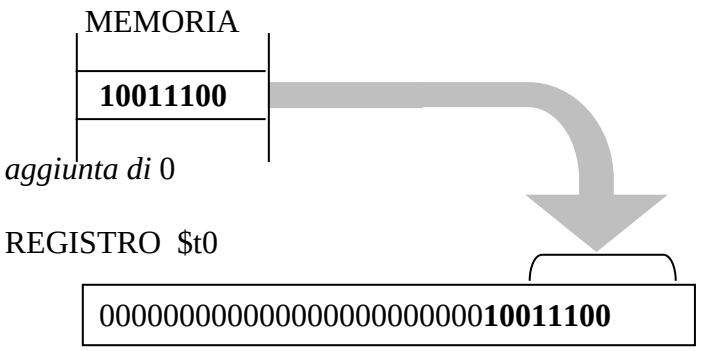
ESEMPIO: **Load BYTE**

lb $t0, 50 ($s1)



ESEMPIO: **Load BYTE *unsigned***

lbu $t0, 50 ($s1)



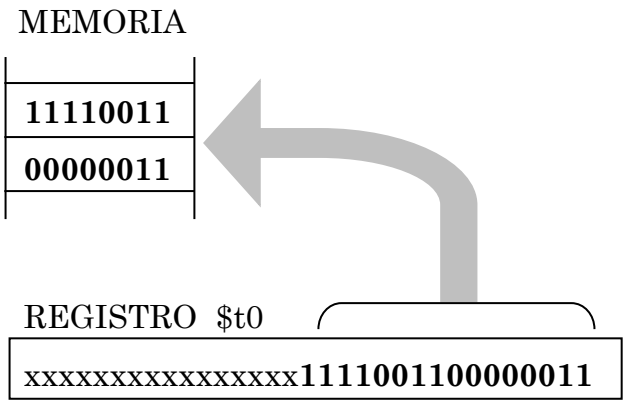
STORE per Byte e *half word*

- **sb** (*store Byte*) Esempio: sb $t0, 50 ($s1) legge gli 8 bit meno significativi del registro $t0 e li scrive in una locazione di Memoria.
- **sh** (*store half word*) Esempio: sh $t0, 50 ($s1) legge i 16 bit meno significativi del registro $t0 e li scrive in due locazioni di Memoria consecutive.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO: **Store HALF WORD**

sh $t0, 50 ($s1)



Si può notare che nel caso delle istruzioni *store Byte* e *store half word* non è necessario il completamento della sequenza da scrivere perché una locazione di Memoria è un registro di 8 bit, quindi per la scrittura di un Byte si occupa una locazione e per *half word* si occupano due locazioni senza bit vuoti da riempire.

Notiamo anche che per *load word* non si ha una corrispondente istruzione per il Tipo di dato *unsigned* in quanto la scrittura di una *parola* (32 bit) in un registro occupa tutti i 32 bit disponibili, quindi non si ha la necessità di distinguere tra i due tipi di completamento.

Per queste istruzioni si mantiene la Sintassi, il Formato e la Modalità di Indirizzamento delle istruzioni *lw* e *sw* relative a sequenze di 32 bit, che si richiamano brevemente qui di seguito.

LINGUAGGIO ASSEMBLY MIPS: vincoli sintattici

- Codice Operativo che specifica l'operazione,
- dopo il Codice Operativo viene indicato l'**indirizzo di un registro del processore**,
- separato da una virgola segue un numero intero detto **OFFSET** (spiazzamento),
- in parentesi è indicato l'**indirizzo del registro** detto **REGISTRO BASE** che contiene un **indirizzo di Memoria.**

L'Offset ed il Registro Base vengono utilizzati per il calcolo dell'indirizzo di accesso in Memoria che individua una locazione in cui è possibile memorizzare gli 8 bit, che sono sufficienti nel caso di trasferimento di un dato rappresentano da un Byte. Nel caso di trasferimento di un dato rappresentato con 16 bit (*half word*), sono necessarie due locazioni consecutive per la relativa memorizzazione.

In base all'approccio *Big-endian*, l'indirizzo di accesso in Memoria è sempre relativo agli 8 bit più significativi (Byte più significativo) della sequenza binaria che rappresenta il dato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Formato e Indirizzamento per trasferimento di Byte e half word

Il Formato suddivide la sequenza binaria che rappresenta l'istruzione in **LINGUAGGIO MACCHINA** in sottosequenze, dette CAMPI, di lunghezza e posizione fissata i cui contenuti forniscono specifiche informazioni determinate univocamente dalle regole sintattiche dell'istruzione. Il formato utilizzato dalle istruzioni di trasferimento è:

op: 6 bit, campo del *Codice Operativo* presente in tutte le istruzioni, indica l'operazione da eseguire ed il Formato dell'istruzione;

rs: 5 bit, indirizzo del REGISTRO BASE indicato in parentesi;

rt: 5 bit, indirizzo del registro del processore in cui scrivere (*load*) o leggere (*store*) il dato;

intero immediato: 16 bit, contiene l'OFFSET in Notazione in complemento a 2.

I *Codici Operativi* in Notazione decimale e binaria corrispondenti alle istruzioni precedentemente elencate sono:

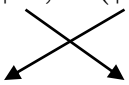
lb	32	100000	lh	33	100001
sb	40	101000	sh	41	101001
lbu	36	100100	lhu	37	100101

L'ordine degli indirizzi dei registri che figurano nell'istruzione Assembly viene invertito nell'istruzione tradotta in Linguaggio Macchina, quindi il *registro base* indicato in parentesi viene riportato nel campo *rs* del Formato I.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESEMPIO lb $t1, 7 ($s0)



op	*rs*	*rt*	*intero immediato*
32	16	9	7

la cui rappresentazione binaria è:

100000	10000	01001	0000000000000111

e quindi la sequenza dei 32 bit in Linguaggio Macchina è:

10000010000010010000000000000111

MODALITÀ DI INDIRIZZAMENTO

L'Offset ed il Registro Base vengono utilizzati per il calcolo dell'**indirizzo di accesso in Memoria** da cui inizia la rappresentazione del dato da trasferire. Entrambe le istruzioni *Load* e *Store* utilizzano la stessa Modalità di Indirizzamento per il calcolo di tale indirizzo detta:

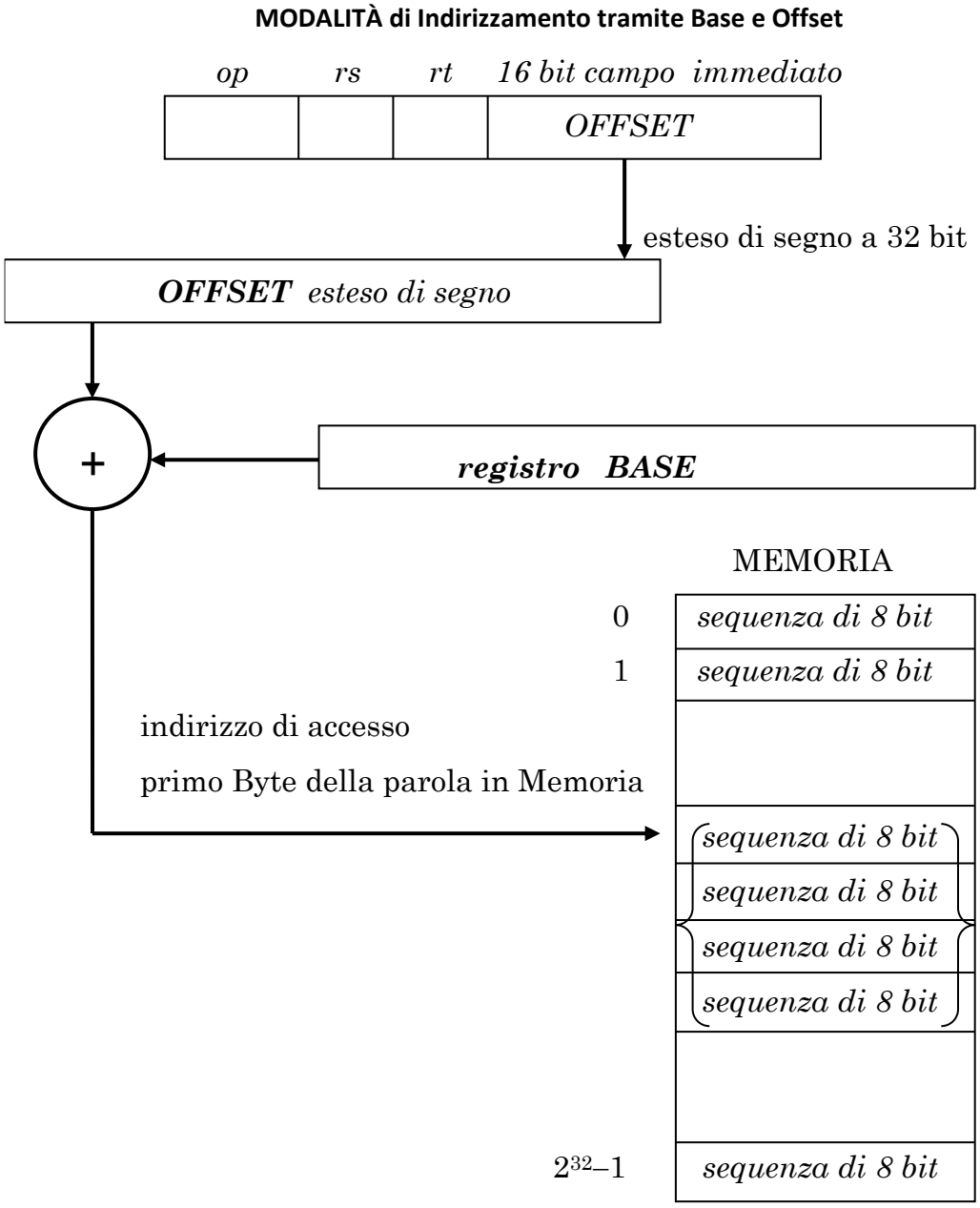
INDIRIZZAMENTO TRAMITE BASE E OFFSET

Indirizzo di Memoria (primo Byte del dato da trasferire) = contenuto del *Registro Base* + *Offset* **esteso di segno** a 32 bit

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La seguente figura fornisce una rappresentazione schematica della Modalità di Indirizzamento tramite Base e Offset:



L'estensione del segno del valore dell'*Offset* contenuto nel *campo immediato* di 16 bit è necessaria in quanto l'addizione è eseguita dalla Unità Aritmetico-Logica (ALU) che richiede operandi di 32 bit.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'estensione del segno aggiunge i bit a sinistra della sequenza ponendo il loro valore uguale a quello del bit di segno. In base alle proprietà della Notazione in complemento a 2, l'estensione del segno non cambia il valore rappresentato.

Quando l'Offset ha valore 0 l'indirizzo di accesso alla Memoria coincide con l'indirizzo Base contenuto nel registro Base.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Tipi di dato primitivi per gli interi

Le **istruzioni Assembly MIPS di trasferimento dati** per sequenze di 8 bit (Byte), 16 bit (mezza parola) e 32 bit (parola) consentono la gestione dei principali Tipi di dato primitivi per interi e caratteri disponibili nei linguaggi ad Alto Livello. La seguente tabella riporta le dimensioni dei Tipi interi relative al Linguaggio Java.

In Java la dimensione è fissa, in altri Linguaggi può dipendere dal tipo di piattaforma. In C e C++ la funzione *sizeof*(*int*) consente di conoscere la dimensione implementata.

Tipo	bit	minimo	massimo
char	8 ASCII	–128	127
char	16 Unicode	0	65535
byte	8	–128	127
short	16	–32768	32767
int	32	circa –2 miliardi	circa 2 miliardi
long	64	circa –9 miliardi di miliardi	circa 9 miliardi di miliardi

I Linguaggi C e C++ dispongono anche dei corrispondenti Tipi di dato primitivi *unsigned* non presenti in Java dove vengono gestiti mediante metodi della classe *Integer*. La seguente tabella mostra le dimensioni dei Tipi *unsigned* corrispondenti ai Tipi prima riportati.

Tipo	bit	minimo	massimo
unsigned char	8	0	255
unsigned short	16	0	65535
unsigned int	32	0	circa 4 miliardi
unsigned long	64	0	circa 18 miliardi di miliardi

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I linguaggi ad Alto Livello consentono di effettuare la conversione da un Tipo di dato in un altro Tipo di dato che utilizza una sequenza binaria di lunghezza diversa.

Una *widering conversion* (conversione di ampliamento) aumenta il numero di bit della rappresentazione. In questo caso tutti i possibili valori dei dati originari sono rappresentabili.

Una *narrowing conversion* (conversione di restringimento) diminuisce il numero di bit della rappresentazione. In questo caso il nuovo Tipo di dato non consente la rappresentazione di tutti i possibili valori dei dati originari.

Nel Linguaggio Java la conversione di ampliamento che porta dal Tipo di dato *byte* al Tipo di dato *char* aggiunge degli 0 nelle posizioni più significative per ottenere la sequenza binaria di 16 bit (lunghezza del codice Unicode). Quindi, poiché il Tipo di dato *byte* è rappresentato in Notazione in complemento a 2, nel caso di numeri negativi la nuova rappresentazione a 16 bit ha un valore diverso da quello rappresentato dalla sequenza originaria a 8 bit, poiché è necessaria l'*estensione del segno* per mantenere il valore invariato.

Le seguenti Tabelle riassumono per il sottoinsieme di istruzioni presentate le regole sintattiche del Linguaggio Assembly e del Liguaggio Macchina MIPS, e la relativa Modalità di Indirizzamento.

I valori dei campi del Formato in Linguaggio Macchina sono espressi mediante la notazione decimale per i valori numerici, e la notazione simbolica per gli indirizzi dei registri del processore.

Vengono inoltre ricordati gli indirizzi dei registri del processore MIPS utilizzati con notazione simbolica nelle istruzioni Assembly e, per completezza, anche i registri riservati al Software di Sistema.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

		6 bit	5 bit	5 bit	5 bit	5 bit	6 bit	esempio
istruzione	*Tipo*	*op*	*rs*	*rt*	*rd*	*shamt*	*funct*	
add	R	0	$s1	$s2	$t0	0	32	add $t0, $s1, $s2
sub	R	0	$s1	$s2	$t0	0	34	sub $t0, $s1, $s2
and	R	0	$s1	$s2	$t0	0	36	and $t0, $s1, $s2
or	R	0	$s1	$s2	$t0	0	37	or $t0, $s1, $s2
slt	R	0	$s1	$s2	$t0	0	42	slt $t0, $s1, $s2
sll	R	0	0	$s2	$t0	7	0	sll $t0, $s2, 7
srl	R	0	0	$s2	$t0	7	2	srl $t0, $s2, 7
lw	I	35	$s3	$t0	–9			lw $t0, –9 ($s3)
sw	I	43	$s3	$t0	–9			sw $t0, –9 ($s3)
beq	I	4	$t0	$s3	istruzioni da saltare			beq $t0, $s3, *nome*
bne	I	5	$t0	$s3	istruzioni da saltare			bne $t0, $s3, *nome*
j	J	2	indirizzo di salto incompleto					j *nome*
jal	J	3	indirizzo di salto incompleto					jal *nome*
jr	R	0	$ra	0	0	0	8	jr $ra
addu	R	0	$s1	$s2	$t0	0	33	addu $t0, $s1, $s2
subu	R	0	$s1	$s2	$t0	0	35	subu $t0, $s1, $s2
sltu	R	0	$s1	$s2	$t0	0	43	sltu $t0, $s1, $s2
addi	I	8	$s3	$t0	–9			addi $t0, $s3,–9
slti	I	10	$s3	$t0	–9			slti $t0, $s3,–9
andi	I	12	$s3	$t0	9			andi $t0, $s3, 9
ori	I	13	$s3	$t0	9			ori $t0, $s3, 9
lb	I	32	$s3	$t0	–9			lb $t0, –9 ($s3)
lh	I	33	$s3	$t0	–9			lh $t0, –9 ($s3)
lbu	I	36	$s3	$t0	–9			lbu $t0, –9 ($s3)
lhu	I	37	$s3	$t0	–9			lhu $t0, –9 ($s3)
sb	I	40	$s3	$t0	–9			sb $t0, –9 ($s3)
sh	I	41	$s3	$t0	–9			sh $t0, –9 ($s3)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

istruzione	*Formato Tipo*	*Modalità di Indirizzamento*
add	R	tramite registro
sub	R	tramite registro
and	R	tramite registro
or	R	tramite registro
slt	R	tramite registro
sll	R	tramite registro
srl	R	tramite registro
lw	I	tramite Base e Offset
sw	I	tramite Base e Offset
beq	I	relativo al *Program Counter*
bne	I	relativo al *Program Counter*
j	J	pseudodiretto
jal	J	pseudodiretto
jr	R	tramite registro
addu	R	tramite registro
subu	R	tramite registro
sltu	R	tramite registro
addi	I	immediato
slti	I	immediato
andi	I	immediato
ori	I	immediato
lb	I	tramite Base e Offset
lh	I	tramite Base e Offset
lbu	I	tramite Base e Offset
lhu	I	tramite Base e Offset
sb	I	tramite Base e Offset
sh	I	tramite Base e Offset

L'elenco completo delle istruzioni dell'Architettura MIPS è riportato nell'Appendice B del libro: D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

INDIRIZZI DEI REGISTRI del processore MIPS

Notazione decimale e corrispondente notazione simbolica MIPS

Registri utilizzati per i dati:

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$t0	8	$s0	16	$t8	24
$t1	9	$s1	17	$t9	25
$t2	10	$s2	18		
$t3	11	$s3	19		
$t4	12	$s4	20		
$t5	13	$s5	21		
$t6	14	$s6	22		
$t7	15	$s7	23		

Registri dedicati alla gestione delle chiamate di procedura che contengono: $a i parametri passati, $v i valori ritornati, $ra l'indirizzo di ritorno. I registri $gp $sp $fp sono utilizzati per la gestione del *call stack.*

simbolo	*indirizzo*	*simbolo*	*indirizzo*	*simbolo*	*indirizzo*
$v0	2	$a0	4	$gp	28
$v1	3	$a1	5	$sp	29
		$a2	6	$fp	30
		$a3	7	$ra	31

Il registro all'indirizzo 0 contiene una costante non modificabile che rappresenta il valore zero rappresentato da 32 bit tutti uguali a 0:

simbolo	*indirizzo*
$zero	0

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Registri riservati al Software di Sistema. Il registro $at è riservato al programma Assembler, i registri $k0 e $k1 al Sistema Operativo:

simbolo	*indirizzo*
$at	1
$k0	26
$k1	27

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- ORACLE Java Documentation, The Java Tutorials, Primitive Data Types
  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).